

EQUIVALENCE OF CFG AND PDA

CFG to PDA

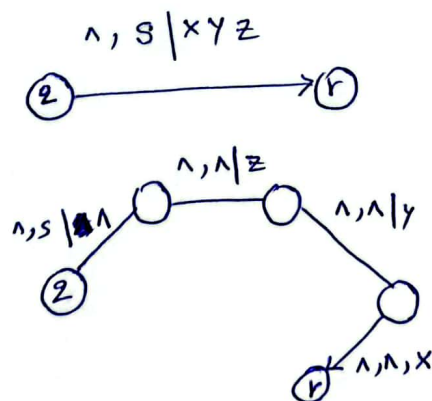
A language is context-free if and only if some Pushdown Automaton recognize it.

Part A

If a language is context free, then some Pushdown Automaton recognize it.

Covert CFG to PDA

1. Place the ^{Start} ~~initial~~ symbol on the stack
2. Repeat the following steps forever.
 - a. If the top of Stack is a variable, nondeterministically select one of the rules for that variable and substitute that variable by the string on the right-hand side of the rule.
 - b. If the top of stack is terminal symbol, read the next symbol from the input and compare it to the stack symbol (terminal symbol). If they match, repeat. If they do not match, reject on this branch of non-determinism.
 - c. If the top of stack is the symbol z_0 , enter the accept state. Doing so accepts the input if it has all been read.



EXAMPLE

2

Convert the following CFG into Pushdown Automaton

$$\begin{aligned} S &\rightarrow AA \mid a \\ A &\rightarrow SA \mid b \end{aligned}$$

Solution:

$$\begin{aligned} S &\rightarrow AA \mid a \\ A &\rightarrow SA \mid b \end{aligned}$$

$$G = (V, \Sigma, S, P)$$

$$V = \{S, A\}$$

$$\Sigma = \{a, b\}$$

S is the start variable

$$P = \{S \rightarrow AA, S \rightarrow a, A \rightarrow SA, A \rightarrow b\}$$

Its equivalent PDA:

$$M = (Q, \Sigma, \Gamma, q_0, Z_0, F, \delta)$$

$$Q = \{q_0, q_1, q_2\} \text{ - Taken always for every CFG}$$

$$\Sigma = \text{terminals in CFG} = \{a, b\}$$

$$\Gamma = (V \cup \Sigma \cup \{Z_0\}) = \{S, A, a, b, Z_0\}$$

q_0 is the start state of PDA

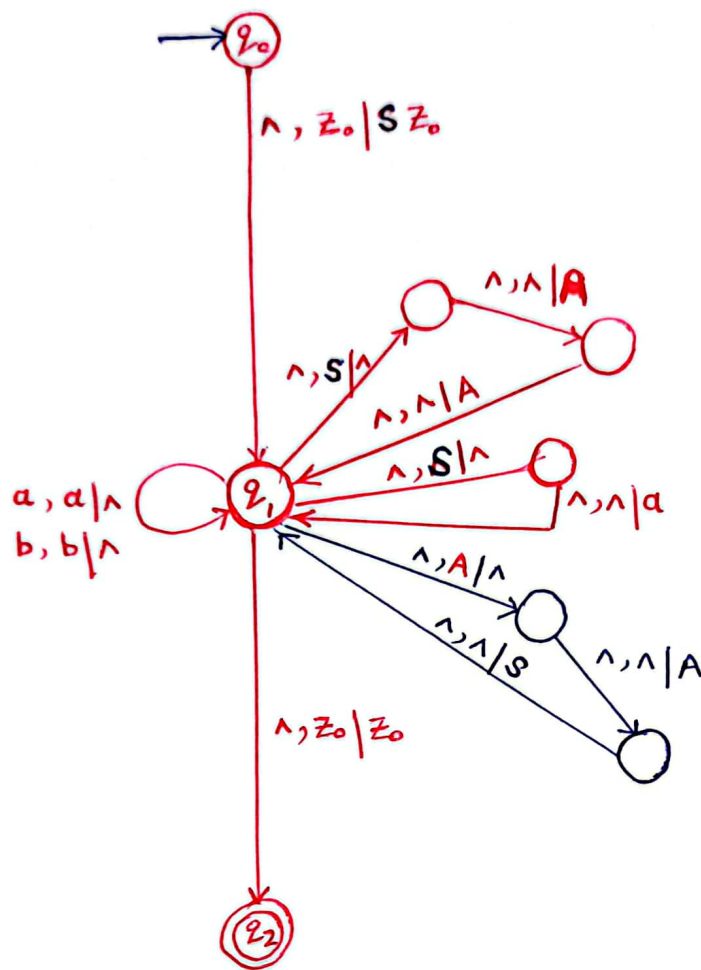
Z_0 is the initial symbol of stack

$$F = \{q_2\} \text{ - Taken always}$$

δ : transition function defined as

State	Input	Stack	Move(v)
q_0	$\wedge$	Z_0	(q_0, SZ_0)
q_1	$\wedge$	S	$(q_1, AA), (q_1, a)$
q_1	$\wedge$	A	$(q_1, SA), (q_1, b)$
q_1	a	a	$(q_1, \wedge)$
q_1	b	b	$(q_1, \wedge)$
q_1	$\wedge$	Z_0	(q_2, Z_0)

TRANSITION DIAGRAM



$\lambda, z_0 | S z_0$: Having z_0 on the top of stack, Push S , (the start Non-terminal) on to the stack, without seeing the input symbol

$\lambda, z_0 | z_0$: if we have z_0 on top of the stack and λ is input symbol (i.e., complete input has been consumed and λ is found after the last input symbol, nothing to do with stack symbol).

(Having S on top of stack)

$\lambda, S | \lambda$: Pop S from the stack without reading input symbol

$\lambda, \lambda | A$: Push A on the stack without reading input symbol, without seeing the stack symbol.

$a, a | \lambda$: if a is the input symbol and top of the stack is a , then consume a from the input symbol and pop a from the stack

Suppose we have a word $w = abbabb$

To find whether the given word $w = \text{abbabb}$ is accepted by PDA or not, we execute this word on PDA as shown below:

As this PDA is non-deterministic, so first we derive the word from CFG using Left most derivation (to avoid non-deterministic nature which may lead to rejection).

 $w = abbabbb$

$(z_0, abbabb, z_0) \vdash (z_1, abbabb, SZ_0)$
 $\vdash (z_1, abbabb, z_0)$ pop S
 $\vdash (z_1, abbabb, AZ_0)$ push A
 $\vdash (z_1, abbabb, AA Z_0)$ push A
 $\vdash (z_1, abbabb, A Z_0)$ pop A
 $\vdash (z_1, abbabb, AA Z_0)$ push A
 $\vdash (z_1, abbabb, SAA Z_0)$ push S
 $\vdash (z_1, abbabb, aAA Z_0)$ replace S by its definition a
 $\vdash (z_1, abbabb, AA Z_0)$ matching of a (consonant)
 $\vdash (z_1, bbabb, bAZ_0)$ (replace A by b) input symbol and pop a from the stack
 $\vdash (z_1, bbabb, AZ_0)$ matching b
 $\vdash (z_1, babb, z_0)$ pop A
 $\vdash (z_1, babb, AZ_0)$ push definition in reverse direction
 $\vdash (z_1, babb, SA Z_0)$
 $\vdash (z_1, babb, AA Z_0)$
 $\vdash (z_1, babb, bAA Z_0)$
 $\vdash (z_1, abb, AA Z_0)$ match
 $\vdash (z_1, abb, AA Z_0)$ match
 $\vdash (z_1, abb, SAA)$
 $\vdash (z_1, abb, aAA Z_0)$
 $\vdash (z_1, bb, AA Z_0)$ match

$S \Rightarrow A A$
 $\Rightarrow S A A$ (using $A \rightarrow SA$)
 $\Rightarrow a A A$ (using $S \rightarrow a$)
 $\Rightarrow a b A$ (using $A \rightarrow b$)
 $\Rightarrow a b S A$ (using $A \rightarrow SA$)
 $\Rightarrow a b A A A$ ($S \rightarrow AA$)
 $\Rightarrow a b b A A$ ($A \rightarrow b$)
 $\Rightarrow a b b S A A$ ($A \rightarrow SA$)
 $\Rightarrow a b b a A A$
 $\Rightarrow a b b a b A$
 $\Rightarrow a b b a b b$
 $\Rightarrow a b b a b b$ Accepted

(replace A by SA in rev.)
 A $\rightarrow$ SA
 Pop Push Push
 (reverse push)

word is accepted
 PDA.